UNIVERSIDAD NACIONAL DEL CENTRO DEL PERÚ
FACULTAD DE INGENIERÍA DE SISTEMAS
DEPARTAMENTO ACADÉMICO DE INGENIERÍA DE SISTEMAS
PROGRAMA DE INGENIERÍA DE SISTEMAS

GUÍA PRÁCTICA SEMANA 07 – 2° PARTE

Asignatura: Desarrollo de Aplicaciones Web (IS093A)
Unidad I: Desarrollo Web Frontend
Tema: React Hooks: useState, useEffect, useContext, useRef, useReducer, useCallback, useMemo y Hooks Personalizados

Alumno: _____

OBJETIVO DE LA PRÁCTICA

Debe desarrolla un **Dashboard de Gestión con Estado Complejo** que cumpla:

1. useReducer para estado de lista (add, remove, filter, sort) + useState para UI local.
2. useContext para tema claro/oscuro o preferencias de usuario.
3. useEffect para sincronizar preferencias con localStorage y cleanup.
4. useMemo para lista filtrada/ordenada, useCallback para handlers de botones.
5. useRef para auto-focus en input o acceso a nodo DOM sin re-render.
6. Hook personalizado useLocalStorage o useDebounce reutilizable.

Roles Técnicos

Rol	Responsabilidad
Arquitecto de Estado	useReducer, acciones, reducer function, inmutabilidad
Ingeniero de Efectos & Contexto	useEffect, useContext, sync con storage, cleanup
Optimizador de Rendimiento	useMemo, useCallback, useRef, análisis Profiler
QA & Hook Validator	Reglas de Hooks, ESLint warnings, documentación técnica, custom hook

Entregable 1: En su repositorio de Github

- Repositorio GitHub o ZIP con: package.json, src/, README.md
- README.md debe incluir:
  - Diagrama de flujo de estado y ciclo de render
  - Justificación técnica de cada hook utilizado (¿por qué no useState simple?)

Entregable 2: Validación Final

1. Ejecutar npm run dev y verificar cero warnings de react-hooks/exhaustive-deps en ESLint.
2. Abrir React DevTools Profiler: grabar interacción, identificar componentes que se re-renderizan y validar que useMemo/useCallback reducen commits.
3. Verificar que useEffect tenga cleanup activo (simular navegación rápida o desmontaje).
4. Evidenciar y explicar con sus propias palabras cada una de las validaciones con capturas de pantalla.
5. Subir a GitHub así mismo subir en rar su proyecto en el ADESA.

Rúbrica de Evaluación

Criterio	**Peso**
Uso correcto de useReducer + manejo de estado complejo	20%
useContext + useEffect (sync, cleanup, dependencias)	20%
useMemo/useCallback aplicados estratégicamente (sin overuse)	20%
useRef + Hook personalizado funcional y reutilizable	20%
Validación Profiler/ESLint, documentación	20%

CÓDIGO BASE DE LA GUÍA PRÁCTICA (Refactorizar)

src/contexts/ThemeContext.jsx

jsx

```jsx
import { createContext, useState, useEffect } from 'react'

export const ThemeContext = createContext()

export const ThemeProvider = ({ children }) => {
  // TODO: Inicializar estado con localStorage o valor por defecto
  const [theme, setTheme] = useState('light')

  // TODO: useEffect para aplicar clase al <body> y persistir en localStorage
  // TODO: Función toggleTheme expuesta via value

  return (
    <ThemeContext.Provider value={{ theme, toggleTheme }}>
      {children}
    </ThemeContext.Provider>
  )
}
```

src/hooks/useTasksReducer.js

js

```js
// TODO: Definir tipos de acción: ADD, REMOVE, FILTER, SORT
const initialState = { items: [], filter: 'all', sort: 'asc' }

function tasksReducer(state, action) {
  switch (action.type) {
    // TODO: Implementar casos inmutables
    default: return state
  }
}

export default tasksReducer
```

src/hooks/useLocalStorage.js (Hook Personalizado)

```js
import { useState, useEffect } from 'react'

// TODO: Implementar hook que sincronice estado con localStorage
// Recibe key y defaultValue, retorna [value, setValue]
export const useLocalStorage = (key, defaultValue) => {
  // ...
}
```

src/components/TaskList.jsx

```jsx
import { useMemo, useCallback, useRef } from 'react'

const TaskList = ({ tasks, onRemove, onToggle }) => {
  const inputRef = useRef(null)

  // TODO: useMemo para filtrar/ordenar tareas sin recalcular en cada render
  const processedTasks = useMemo(() => {
    // ...
  }, [tasks])

  // TODO: useCallback para estabilizar handlers pasados a hijos
  const handleRemove = useCallback((id) => {
    onRemove(id)
  }, [onRemove])

  // TODO: useEffect para focus al input cuando se añade tarea
  // ...

  return (
    <ul>
      {processedTasks.map(task => (
        <li key={task.id}>
          <span>{task.title}</span>
          <button onClick={() => handleRemove(task.id)}>Eliminar</button>
        </li>
      ))}
      <input ref={inputRef} placeholder="Nueva tarea..." />
    </ul>
  )
}

export default TaskList
```

src/App.jsx

```jsx
import { useReducer, useContext } from 'react'
import tasksReducer from './hooks/useTasksReducer'
import { ThemeProvider, ThemeContext } from './contexts/ThemeContext'
import TaskList from './components/TaskList'

const Dashboard = () => {
  const [state, dispatch] = useReducer(tasksReducer, { items: [], filter: 'all', sort: 'asc' })
  const { theme, toggleTheme } = useContext(ThemeContext)

  // TODO: Conectar dispatch a handlers, renderizar TaskList y controles
  return (
    <div className={`dashboard ${theme}`}>
      <button onClick={toggleTheme}>Cambiar Tema</button>
      <h1>Gestor de Tareas</h1>
      {/* TODO: Renderizar lista y formularios con dispatch */}
    </div>
  )
}

export default function App() {
  return (
    <ThemeProvider>
      <Dashboard />
    </ThemeProvider>
  )
}
```

NOTAS TÉCNICAS CLAVE

Concepto	Regla / Aplicación
Reglas de Hooks	Solo en nivel superior, nunca en loops/condiciones/funciones anidadas. Solo en componentes funcionales o custom hooks.
useEffect	Array de dependencias determina cuándo se ejecuta. Cleanup (return () => {}) evita memory leaks y race conditions.
useContext	Ideal para evitar prop drilling, pero no para estado que cambia frecuentemente (dispara re-render en todos los consumidores).
useReducer	Preferible a useState cuando el estado es complejo, tiene múltiples sub-valores o la lógica de actualización depende del estado anterior.

useMemo / useCallback	Solo optimizar si hay bottleneck medible. Memoizar objetos/funciones pasados como props a hijos React.memo.
useRef	Persiste entre renders sin disparar re-render. Ideal para acceso DOM, timers, o valores mutables que no afectan UI.
Profiler	React DevTools → Profiler → Record. Analiza render time, commits, why did this render?. No optimizar sin medir.